QCLog

April 28, 2026

Abstract

QCLog is a **Q**uick, **C**ontest-oriented amateur radio **L**ogger. QCLog prioritizes speed and convenience over features. It's intended to run reliably and provide a minimal feature set, meeting the essential requirements for contest logging:

- Exchange logging (callsign/class/section by default, but customizable)
- Dupe checking
- Band/mode synchronization from the rig
- Current operator
- Network sync between multiple logging stations

Other features — common in (contest) loggers — are excluded to eliminate dependencies, simplify the UI, and avoid annoying behavior like unreadable text in dark-mode desktop themes.

Contents

1 Logging with QCLog **2**
1.1 Status . . . 3
1.2 Advanced Usage . . . 4
1.3 Networking . . . 5
1.3.1 Resync and Network Traffic . . . 6

2 Command Line **6**
2.1 ADIF export for POTA . . . 6
2.2 Cabrillo export . . . 6

3 Common Setups **7**
3.1 Standalone . . . 7
3.2 Digital Modes (Full FLDigi integration) . . . 8
3.3 FLRig-only . . . 8
3.4 Rigctld . . . 8
3.5 Hamlib (TERRIBLE) . . . 9

4 Installation and Dependencies **9**
4.1 Flatpak . . . 9
4.2 Windows . . . 10

5 Extending **10**

6 The Future **10**

1 Logging with QCLog

The QCLog exchange field(s) are configurable, but field day is the default and will be used as an example throughout this guide. See Figure 1 for the basic QCLog interface.

Figure 1: QCLog interface

The main focus of the QCLog UI is to be simple and consistent. To wit:

- Escape *always* clears all input and returns the cursor to callsign input.
- Tab cycles through input fields in order (callsign/class/section).
- Enter submits a log entry, rejecting duplicate or incomplete entries
- Ctrl-enter force logs *whatever* is currently entered, no matter what.
- The Mode/Band/Frequency/Operator fields are set by right-clicking to spawn a dialog. Note that Mode/Band/Frequency will be auto-populated by appropriate rig integration, but operator must be manually set (or specified by command line options at startup).

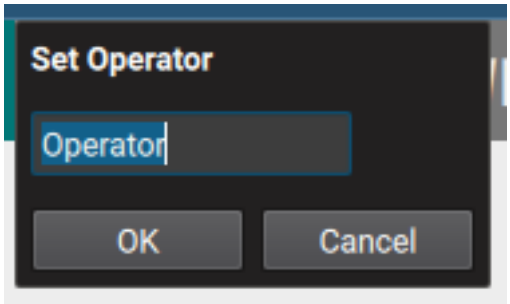

Figure 2: "Set Operator" dialog when right-clicking on "Operator" field

See Figure 2 for the "Set Operator" dialog. Depending on window manager behavior, the full dialog might not appear (see the documented **BUG**).

While a tiling window manager is not required, QCLog is intended to co-operate very well with a tiling window manager (or tiling features of another type of window manager). Figure 3 shows QCLog tiled with FLDigi (a major intended use-case).

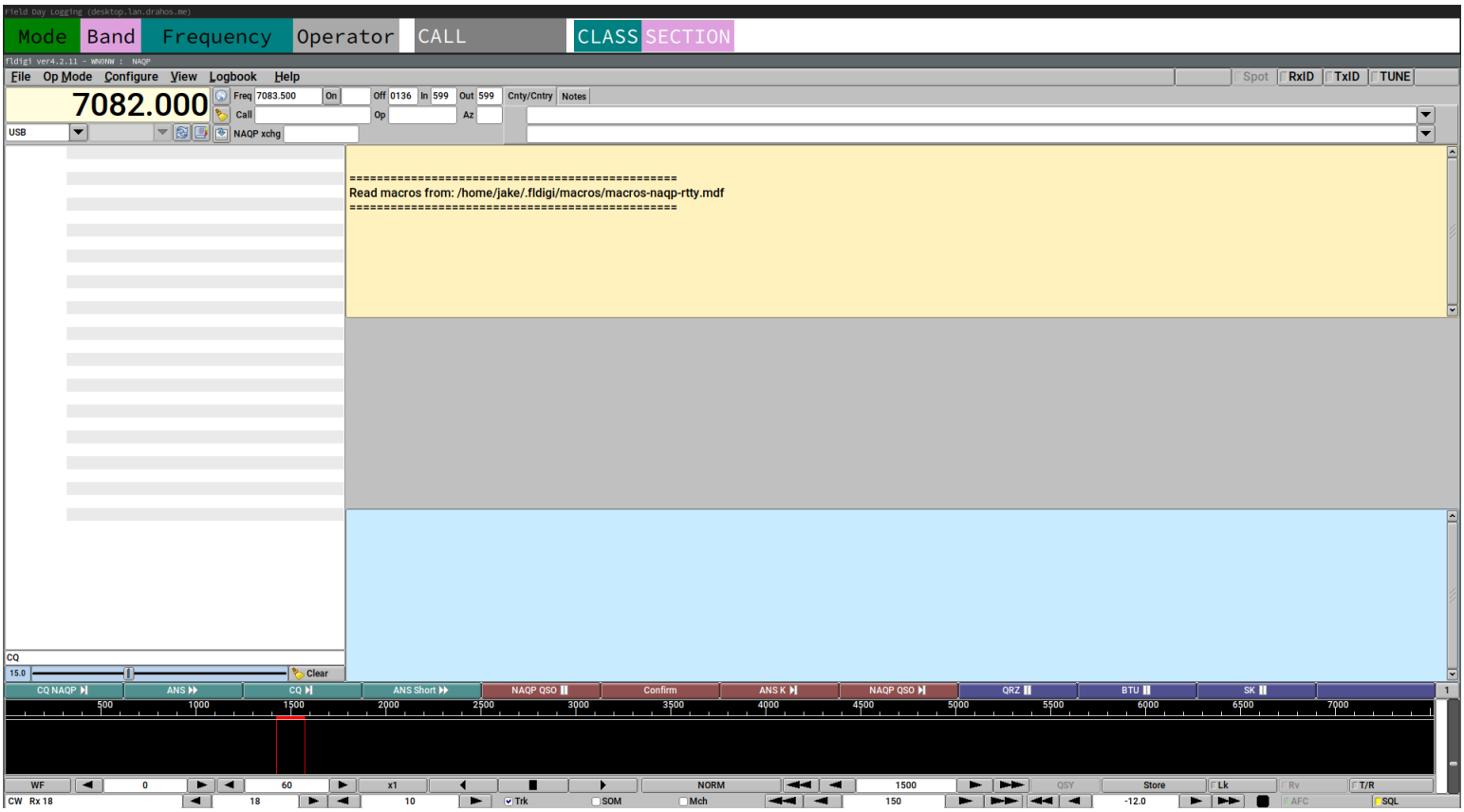

Figure 3: QCLog tiled with FLDigi

The intent is that it will look and feel like a native extension of any other program that might be in use at the time. QCLogs is to accept log entries, flag dupes, and that's all.

BUG: QCLog doesn't currently supply the right window hints to fully float the dialog when right clicking to edit a field, so the dialog is bounded to the height of the QCLog main window in i3. The field won't be visible, but it still gets keyboard focus, so operator etc. can be manually input.

A note on modifying/deleting entries. You can't. But, you can always just type "DELETE THE PREVIOUS QSO" or "PREV SECTION SHOULD BE IL" into the callsign field, then ctrl-enter to force log it. Clean it up after exporting Cabrillo; it should be easy to catch since such "notes" will pretty obviously fail any callsign validation or otherwise stick out like a sore thumb.

1.1 Status

Figure 4: Status bar of QCLog

The title bar provides log confirmation and basic status. Refer to Figure 4. The status items are:

- "Field Day Logging" — Name taken from interface definition.
- "(desktop.lan.drahos.me)" — Station name. Defaults to hostname, but can be overridden with command line options (see Networking for more details).
- "Logged KF0TEST at 03:59:07UTC" — Log confirmation (most recent local QSO).
- "KE0CDT logged by chromebook at 05:42:16UTC" — Most recent remote log (logging station "chromebook" logged callsign "KE0CDT").

Figure 5: Important status message (rig communication lost)

The status in the title bar largely provides confirmation. Important alerts show up in red to the right of the exchange entry, as shown in Figure 5

1.2 Advanced Usage

Logs are stored in a SQLite database. The schema is somewhat flexible for the exchange, but callsign/timestamp and meta-information (band/mode/freq/operator) are required. All QSOs are assigned a UUID when logged, which is used for network synchronization and deduplication. Logs are stored in `$XDG_DATA_HOME/qclog/<log name>.db` (Default `.local/share/qclog/<log name>.db`).

The database schema isn't the simplest thing, but the `log` view contains everything you might need for manual log surgery or export (see Figure 6 for an example of directly querying the log). If you're brave enough to try modifying the underlying logs, the tables are `qsos` and `remote_qsos`. The `log` view is mostly sort of a deduplicated concatenation of these two underlying tables. I usually just wait until after Cabrillo export to clean things up in a text editor.

QCLog also stores "disaster logs" which are append-only and always flushed. A new file is created timestamped at each start and log a plain-text version of the log entry. Should something horrible happen to the SQLite database, you should be able to reconstruct the log from the disaster logs. The disaster logs are stored in the same directory as the database files.

There isn't currently any functionality to actually import a disaster log, but all the data is there. Actually recovering from database corruption or similar disaster is left as an exercise to the reader.

For hints on further log manipulation, see the SQLite documentation, particularly for JSON operations.

```
~ : sqlite3 — Konsole
[ jake@desktop 21:12:08 ~
[ % sqlite3 .local/share/qclog/fieldday.db
SQLite version 3.53.0 2026-04-09 11:41:38
Enter ".help" for usage hints.
sqlite> SELECT * FROM log ORDER BY timestamp ASC LIMIT 10;
```

id	timestamp	band	mode	callsign	exchange	meta
e43a8665-4404-4cbf-9d0f-25fd88b469bb	2025-06-28 18:10:25	20M	USB	N7W	{"class": "2a", "section": "WY"}	{"frequency": "14273000", "operator": "KF0GAV", "station_id": ...
6d7de569-ddfa-4654-89f9-c05386cb4e65	2025-06-28 18:14:14	15M	USB	FIX TIME OF FIRST QSO	{"class":"","section":""}	{"station_id": "1ba529dc-04d6-42df-a617-48ec1fb9118d", ...
55b0a542-9506-457c-8b66-2f63f2ae5ebd	2025-06-28 18:17:15	15M	USB	K5N	{"class":"2a","section":"stx"}	{"station_id": "1ba529dc-04d6-42df-a617-48ec1fb9118d", ...
eb7a0528-cd2e-4d81-b450-c19621bf7d29	2025-06-28 18:31:36	20M	USB	K4ISU	{"class": "2a", "section": "nc"}	{"frequency": "14315000", "operator": "KF0GAV",

Figure 6: Querying the log

1.3 Networking

Networking mostly just works™. QCLog uses UDP broadcasts on port 14300. If you have a system-level firewall, make sure 14300/udp is open since there isn't any `connected/established` state with UDP to make default firewall rules happy. Especially with broadcast UDP.

QCLog networking uses UUIDs to identify different logging stations (operating positions). The station UUID is stored in the database and generated the first time QCLog is launched for a given log name. Additionally, the `-n <station name>` command line parameter can be used to set a station name overriding the hostname. Any station name specified with `-n` is cached in the database and will be used even if no `-n` parameter is supplied on subsequent runs (for a given log name). This should help prevent accidentally reverting the station name to the hostname.

1.3.1 Resync and Network Traffic

QCLog is fairly reasonable when it comes to network traffic. QSOs are broadcast when logged (several hundred bytes at most) and acknowledged when received (several dozen bytes). Stations periodically heartbeat, which includes the station UUID, station name, and local QSO count.

A resync can be triggered if a station realizes it is "missing" QSOs. The resync algorithm is rather primitive, but should scale okay for reasonable QSO counts. Transfers are chunked, so even if a link gets saturated and packets drop, there will still be progress with the resync and the next resync period should start further along.

Overall, QCLog networking seems to work well and be stable on any reasonable WLAN. However, it is *highly* inadvisable to try to run it over a 1200-baud packet link or similar silliness. Attempting to do so would almost certainly lead to interesting results.

2 Command Line

The only mandatory argument is the "log name". Saved settings (and, of course, log entries) are stored under this name. Field Day is the default exchange interface.

`--help` is fully implemented.

2.1 ADIF export for POTA

ADIF export is basically a prototype for POTA. Simply supply `MYCALL:MYPARK` and a mostly POTA-compatible ADIF will probably be exported. See `--help` and `--adif`.

2.2 Cabrillo export

Oh boy this is hacky. There's a raw Python script in the QCLog directory that seems to work. The invocation is somewhat arcane, but Figure 7 is an example of field day Cabrillo export.

Basically, the argument to `-c` is a template for each Cabrillo line. Eache "word" in the space-separated list is a Cabrillo column, and must contain a colon (":") followed by the column width (Cabrillo is technically a fixed-width format). Column templates are literal, unless matching one of the templates. `%C` is replaced with the callsign, while `%M` and `%E` populate from the metadata and exchange, respectively. The echange and meta templates are immediately followed by the name of the JSON field to extract. Of course, all template arguments must include the width specifier.

Knowing this, the example in Figure 7 breaks down to:

- `KF0SYA:13` — Our callsign (literal)
- `3A:2` and `IA:2` — Our exchange (literal)
- `%C:13` — Logged callsign
- `%Eclass:2` — "class" field from exchange JSON

```
qclog : zsh — Konsole
[ jake@desktop 21:25:17 ~/qclog master
 % python3 qclog/logger.py -c 'KF0SYA:13 3A:2 IA:2 %C:13 %Eclass:2 %Esection:3 %Moperator:10'
 fieldday

QSO: 14000 PH 2025-06-28 1810 KF0SYA        3A IA N7W           2A WY  KF0GAV
QSO: 21000 PH 2025-06-28 1814 KF0SYA        3A IA FIX TIME OF FIRST QSO        OPERATOR
QSO: 21000 PH 2025-06-28 1817 KF0SYA        3A IA K5N           2A STX OPERATOR
QSO: 14000 PH 2025-06-28 1831 KF0SYA        3A IA K4ISU         2A NC  KF0GAV
QSO: 21000 PH 2025-06-28 1834 KF0SYA        3A IA K1R           3A NH  JAKE
QSO: 14000 PH 2025-06-28 1841 KF0SYA        3A IA W6TO          2A SJV KF0GAV
QSO: 14000 PH 2025-06-28 1846 KF0SYA        3A IA NE1RI         2A RI  KF0GAV
QSO: 14000 PH 2025-06-28 1849 KF0SYA        3A IA K7HON         3A AZ  KF0GAV
QSO: 14000 PH 2025-06-28 1851 KF0SYA        3A IA AC8XI         1A MI  KF0GAV
QSO: 14000 PH 2025-06-28 1855 KF0SYA        3A IA K4WOK         4A NFL KF0GAV
QSO: 14000 PH 2025-06-28 1900 KF0SYA        3A IA VE7DQU        1B QC  KF0GAV
QSO: 14000 PH 2025-06-28 1922 KF0SYA        3A IA K4JJ          3A GA  KF0GAV
QSO: 21000 PH 2025-06-28 1937 KF0SYA        3A IA W1AW          6F CT  OPERATOR
QSO: 14000 PH 2025-06-28 1939 KF0SYA        3A IA ND9G          1C NFL KF0GAV
QSO: 14000 PH 2025-06-28 1942 KF0SYA        3A IA KK7KNB        1A OR  KF0GAV
QSO: 14000 PH 2025-06-28 1945 KF0SYA        3A IA N7SME         1A WWA KF0GAV
QSO:  7000 PH 2025-06-28 1955 KF0SYA        3A IA K9SH          1D IN  MALCOLM
QSO:  7000 PH 2025-06-28 2012 KF0SYA        3A IA ND4UG         3A ND  MALCOLM
QSO:  7000 PH 2025-06-28 2014 KF0SYA        3A IA W8FY          4F OH  MALCOLM
QSO:  7000 PH 2025-06-28 2015 KF0SYA        3A IA W9I           2A WI  MALCOLM
QSO:  7000 PH 2025-06-28 2033 KF0SYA        3A IA W8TNO         3A NM  MALCOLM
QSO:  7000 PH 2025-06-28 2034 KF0SYA        3A IA K9SS          3A IL  MALCOLM
QSO:  7000 PH 2025-06-28 2042 KF0SYA        3A IA W9VRC         5A WI  MALCOLM
QSO:  7000 PH 2025-06-28 2044 KF0SYA        3A IA W9EBO         1D WI  MALCOLM
QSO:  7000 PH 2025-06-28 2046 KF0SYA        3A IA W9PC          1A IN  MALCOLM
QSO:  7000 PH 2025-06-28 2053 KF0SYA        3A IA KB9OHY        4A IN  MALCOLM
QSO: 14000 PH 2025-06-28 2101 KF0SYA        3A IA K2PUT         3A ENY KF0GAV
QSO:  7000 PH 2025-06-28 2102 KF0SYA        3A IA K9EAM         4A WI  MALCOLM
```

Figure 7: Exporting Cabrillo

- `%Esection:3` — "section" field from exchange JSON
- `%Moperator:10` — "operator" field from metadata JSON
- `fieldday` — Log name (not part of the `-c` argument)

3 Common Setups

3.1 Standalone

Of course, it's completely possible to use QCLog without any rig integration. Band/mode/frequency can be initially populated with command-line flags and changed at runtime by right-clicking on the field.

This is not recommended for obvious reasons. Sorting out 3 hours of logging under the wrong band/mode at 8am on field day sunday is a headache.

3.2 Digital Modes (Full FLDigi integration)

TL;DR: If you already have working, mostly-default FLDigi, just run with `--fldigi` and `--flrig` to enable the FLDigi integration and get band/mode from FLRig.

When running FLDigi with mostly-default rig control settings settings, FLRig takes full control of the rig. Both FLDigi and FLRig have partially-documented XMLRPC interfaces. QCLog will query FLRig for rig data to populate band/mode/frequency. The FLDigi integration simply auto-populates QCLog with any callsign selected in FLDigi. Before starting QCLog, start FLDigi and configure it to use FLRig (either by manually starting FLRig before FLDigi or doing whatever magic lets FLDigi run/manage its own FLRig instance).

Figure 8 shows an example of the FLDigi integration. When you click on a callsign in the FLDigi text windows, it will populate FLDigi's callsign logging field. QCLog monitors that field over the XMLRPC interface (when running with `--fldigi`), and will auto-populate its callsign input. This will trigger a dupe warning as appropriate.

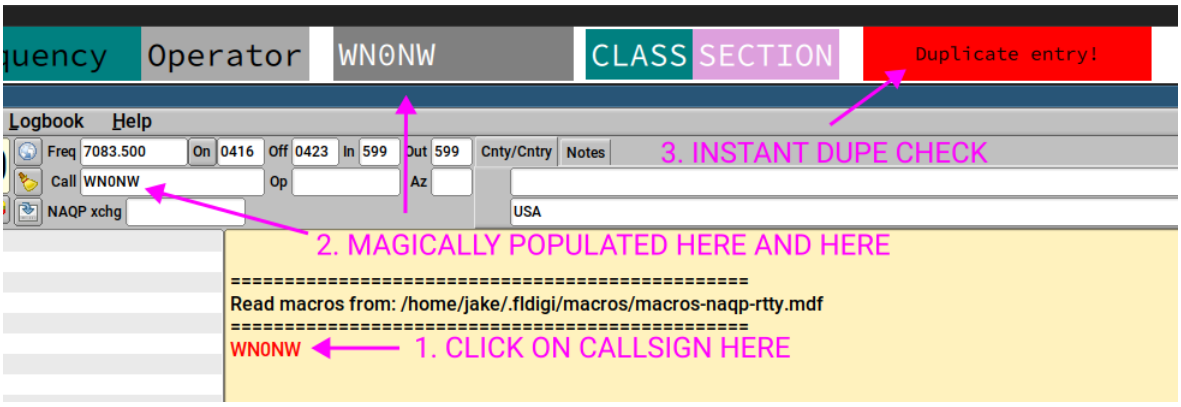

Figure 8: Dupe checking with FLDigi integration

FLRig integration is enabled with `--flrig` and FLDigi with `--fldigi`. There are no arguments for either option, since FL(Rig|Digi) always use the same port for XMLRPC.

3.3 FLRig-only

FLRig can be run independetly of FLDigi. If you can figure out how to run with `--fldigi` and `--flrig`, you can probably figure out how to use only `--flrig`.

3.4 Rigctld

FLRig can sometimes be not-so-stable, or you might not have any FL.* installed. `rigctld` is installed as part of Hamlib and accepts the same arguments as `rigctl`, plus specifying a TCP port to listen on. Simply start `rigctld` before QCLog with all other necessary arguments, then run QCLog with `--rigctld <rigctld listen port>`.

3.5 Hamlib (TERRIBLE)

DO NOT USE. HORRIBLY HORRIBLY BROKEN. Hamlib has a habit of killing the whole process when there are errors. Will probably replace Hamlib with a popen'd rigctl or something.

4 Installation and Dependencies

QCLog is a minimal-dependency Python/Qt6 application. The only dependencies are:

- Qt6 + PySide6
- Python SQLite and XMLRPC

The SQLite and XMLRPC modules are part of the Python standard libraries, so if you have Python, you should *really* have those. Note that the database schema does use some SQLite JSON operators which require a relatively recent version of SQLite3. A certain distribution apparently decided to ship a circa-2019 version of SQLite3 for their 2022 LTS release in the year of our Lord 2025 for some inconceivable reason. This caused problems at field day.

PySide6 is the official Qt binding for Python. Practically all distributions ship it in the repositories. `.deb`-based distributions might not pull in all QPA dependencies when installing PySide6, so unless you already have a good number of Qt or KDE applications/DE, you might get fun errors. `libxcb` really likes to cause problems. It should also be noted that frankendebians are *really* bad at shipping a working distribution of PySide6.

No issues have been observed on OpenSUSE:

```
# zypper in python313-pyside6
```

Will install the only actual dependency of QCLog. Then it just works™.

Obviously, FL(Digi|Rig) or rigctld need to be installed if you want to use those features.

There's no real versioning and absolutely no promise of network interoperability between different commits. To install, simply acquire via git:

```
$ git clone https://github.com/drahosj/qclog
```

With the dependencies installed, it should just...run.

```
$ qclog/main.py logname
```

You can also be cool and set up an alias/symlink/whatever floats your goat. Everything should be relative to the entry point script location, so you don't need to `cd` into the `qclog` directory first.

4.1 Flatpak

It turns out that QCLog builds and runs perfectly well as a flatpak. With the "networking" permission, XMLRPC integration, networking, and rigctld integration all work. Logs are stored in the modified `$XDG_DATA_HOME` per the Flatpak specification.

Actually doing so is left as an exercise for the reader.

4.2 Windows

lol

5 Extending

The front-end is written in QML. The `Interfaces` directory contains the top-level files, which are used to configure the exchange. Look at the existing examples.

The interface can be selected with `-i <interface>` when starting QCLog.

6 The Future

QCLog will not see major further development. It worked for a field day, and that's all that matters. This documentation could quite likely be the final commit.

However, a QCLog-ng is in the works. QCLog-ng will keep the same frontend, but otherwise be a C++ rewrite of the logger core for better maintainability and further reduced dependencies.